A Practical Course in CUDA

Part I — The Main Topics, Simply

June 2026

Abstract

This is the first installment of a CUDA course. The goal here is *breadth, not depth*: introduce each of the core topics simply, with just enough code and intuition to form a mental model. Later installments will elaborate any topic on request — performance tuning, shared-memory algorithms, streams, and so on. If you read this once, you should understand what a kernel is, how threads are organized, how memory works, and how to write and launch a first real program.

1 What CUDA is, and why GPUs are different

CUDA is NVIDIA's platform for running general-purpose code on the GPU. The key mental shift from CPU programming is **throughput over latency**. A CPU has a few powerful cores optimized to finish *one* task quickly. A GPU has thousands of simple cores optimized to run the *same operation over huge amounts of data* at once — the "Single Instruction, Multiple Threads" (SIMT) model. GPUs win when your problem is *data-parallel*: the same computation applied independently to many elements (adding two large arrays, multiplying matrices, training neural networks).

The big picture. Your program has two sides: the **host** (the CPU and its memory) and the **device** (the GPU and its memory). The host orchestrates: it allocates GPU memory, copies data over, launches GPU functions, and copies results back. The device does the heavy parallel work.

2 The programming model: kernels, threads, blocks, grids

A **kernel** is a function that runs on the GPU. When you launch it, it does not run once — it runs in thousands of parallel copies called **threads**, each executing the same code on different data.

Threads are organized in a two-level hierarchy:

- A **block** is a group of threads that run together, can cooperate, and share fast on-chip memory.
- A **grid** is the collection of all blocks launched for one kernel.

So: a *grid* contains many *blocks*, and each *block* contains many *threads*. Both blocks and threads can be indexed in 1D, 2D, or 3D, which is convenient for vectors, images, and volumes respectively.

3 Thread indexing: who am I?

Because every thread runs the same code, each must figure out *which* data element it is responsible for. CUDA gives every thread built-in coordinates:

- `threadIdx` — the thread's index *within* its block.
- `blockIdx` — the block's index within the grid.
- `blockDim` — the number of threads per block.
- `gridDim` — the number of blocks in the grid.

The canonical 1D formula for a thread's *global* index is:

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
```

This maps the flattened (block, thread) pair to a unique position i in a big array — the single most important line of code in beginner CUDA.

4 The memory hierarchy

Performance in CUDA is mostly about memory. The main spaces, fastest to slowest:

- **Registers** — per-thread, fastest; hold local variables.
- **Shared memory** (`__shared__`) — per-block, on-chip, very fast; threads in a block use it to cooperate and to cache data they reuse.
- **Global memory** — the large GPU DRAM (gigabytes); visible to all threads but relatively slow. This is where your input/output arrays live.
- **Constant / texture memory** — small, read-only, cached; good for values all threads read.

A recurring theme of optimization (covered later) is moving reused data from slow global memory into fast shared memory or registers.

5 Your first kernel: vector addition

The "hello world" of CUDA computes $C = A + B$ for large arrays. Each thread handles one element.

```
// Kernel: runs on the GPU, once per thread.
__global__ void vecAdd(const float* A, const float* B, float* C, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;  // global index
    if (i < n)                  // guard: grid may have extra threads
        C[i] = A[i] + B[i];
}

int main() {
    int n = 1 << 20;                    // ~1M elements
    size_t bytes = n * sizeof(float);
    float *hA, *hB, *hC;                // host pointers (allocate + fill omitted)
    float *dA, *dB, *dC;                // device pointers

    cudaMalloc(&dA, bytes);             // 1. allocate on the GPU
    cudaMalloc(&dB, bytes);
    cudaMalloc(&dC, bytes);

    cudaMemcpy(dA, hA, bytes, cudaMemcpyHostToDevice);  // 2. copy in
    cudaMemcpy(dB, hB, bytes, cudaMemcpyHostToDevice);

    int threads = 256;                              // threads per block
    int blocks  = (n + threads - 1) / threads;      // enough blocks to cover n
```

```
    vecAdd<<<blocks, threads>>>(dA, dB, dC, n);   // 3. launch the kernel

    cudaMemcpy(hC, dC, bytes, cudaMemcpyDeviceToHost);  // 4. copy result out

    cudaFree(dA); cudaFree(dB); cudaFree(dC);     // 5. clean up
    return 0;
}
```

The five steps — *allocate, copy in, launch, copy out, free* — are the skeleton of almost every CUDA program. Note the launch syntax `kernel<<<blocks, threads>>>(...)`: the triple angle brackets specify the grid and block dimensions.

6 Compilation and launch

CUDA source files use the `.cu` extension and compile with **nvcc** (NVIDIA's compiler), which splits host code (handed to your normal C++ compiler) from device code (compiled for the GPU):

```
nvcc vecadd.cu -o vecadd && ./vecadd
```

Kernel launches are *asynchronous*: control returns to the host immediately, and a following `cudaMemcpy` (or `cudaDeviceSynchronize()`) waits for the GPU to finish.

7 The execution model: warps and divergence

Inside the hardware, a block's threads run in groups of **32 called a warp**. All 32 threads of a warp execute the same instruction at the same time (in lock-step). This has one crucial consequence: **warp divergence**. If threads in a warp take different branches of an `if/else`, the hardware must run both paths serially, masking off the inactive threads — cutting performance. Writing code where nearby threads follow the same control flow is a basic performance habit.

8 Cooperation: synchronization and atomics

Threads in a block often need to coordinate:

- `__syncthreads()` — a barrier: every thread in the block waits here until all have arrived. Essential when threads write to shared memory and then read what their neighbors wrote.
- **Atomic operations** (e.g. `atomicAdd`) — let many threads safely update the *same* memory location without race conditions, at some cost. Used for histograms, counters, and reductions.

9 Performance, in one breath

We will go deep later, but the three ideas that explain most GPU performance are:

- **Memory coalescing.** When consecutive threads read consecutive memory addresses, the hardware merges them into one wide transaction. Strided or random access patterns waste bandwidth. Lay out data so thread i touches element i.
- **Occupancy.** Keep enough warps "in flight" so the GPU can hide memory latency by switching to other ready warps. Block size and register/shared usage control this.

- **Arithmetic intensity.** The ratio of compute to memory traffic. Most kernels are *memory-bound*; the path to speed is usually doing more work per byte loaded (e.g. shared-memory tiling in matrix multiply).

10 Correctness and tooling

CUDA calls return error codes — check them (a common wrapper macro calls `cudaGetLastError()` after launches). For measurement and debugging, the standard tools are **Nsight Systems** (timeline / where time goes) and **Nsight Compute** (per-kernel deep profiling), with `compute-sanitizer` for memory and race errors.

11 The road map (what we can elaborate next)

Topic	**What a deep dive would cover**
Shared-memory algorithms	Tiled matrix multiply, stencils, the cooperation pattern in full
Parallel reduction	Summing an array efficiently; warp shuffles
Memory optimization	Coalescing in detail, bank conflicts, vectorized loads
Occupancy & launch config	Choosing block size, register pressure, the occupancy calculator
Streams & concurrency	Overlapping copy and compute, multi-kernel pipelines
Warp-level primitives	`__shfl_*`, cooperative groups, ballot/vote
Libraries	cuBLAS, cuDNN, Thrust, CUB — when not to write your own kernel
Tensor Cores	Mixed-precision matrix-multiply hardware for deep learning

Table 1: Suggested next topics. Tell me which to expand.

Summary. A CUDA program launches a *kernel* as a *grid* of *blocks* of *threads*; each thread computes its global index and works on its own data; data moves explicitly between host and device memory; and performance comes from coalesced memory access, enough occupancy to hide latency, and keeping warps converged. Everything else is elaboration — just ask.